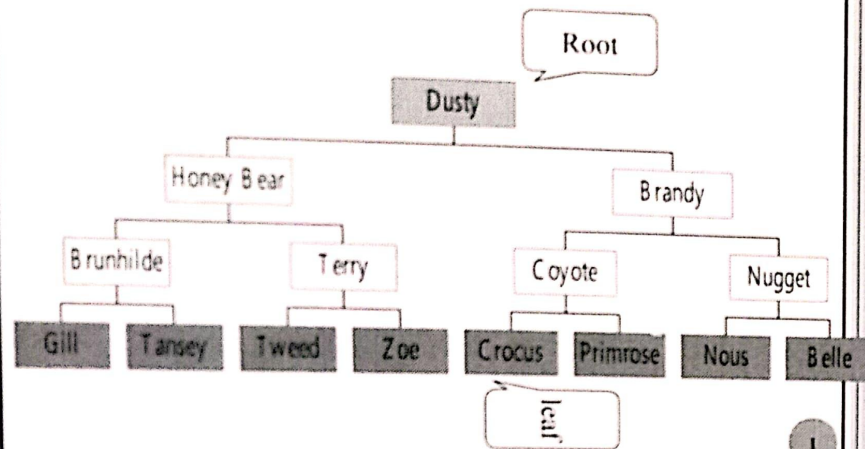


Trees



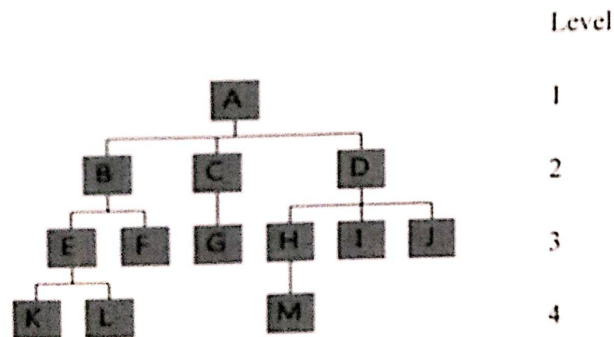
1

Definition of Tree

- A tree is a finite set of one or more nodes such that:
- There is a specially designated node called the root.
- The remaining nodes are partitioned into $n \geq 0$ disjoint sets $T_1, \dots, T_n$, where each of these sets is a tree.
- We call $T_1, \dots, T_n$ the subtrees of the root.

2

Level and Depth



3

Terminology

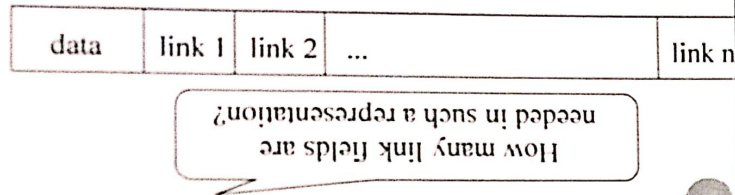
- The degree of a node is the number of subtrees of the node
 - The degree of A is 3; the degree of C is 1.
- The node with degree 0 is a leaf or terminal node.
- A node that has subtrees is the *parent* of the roots of the subtrees.
- The roots of these subtrees are the *children* of the node.
- Children of the same parent are *siblings*.
- The ancestors of a node are all the nodes along the path from the root to the node.

4

REPRESENTATION OF TREES

■ List Representation

- (A (B (E (K, L), F), C (G), D (H (M), I, J)))
- The root comes first, followed by a list of sub-trees



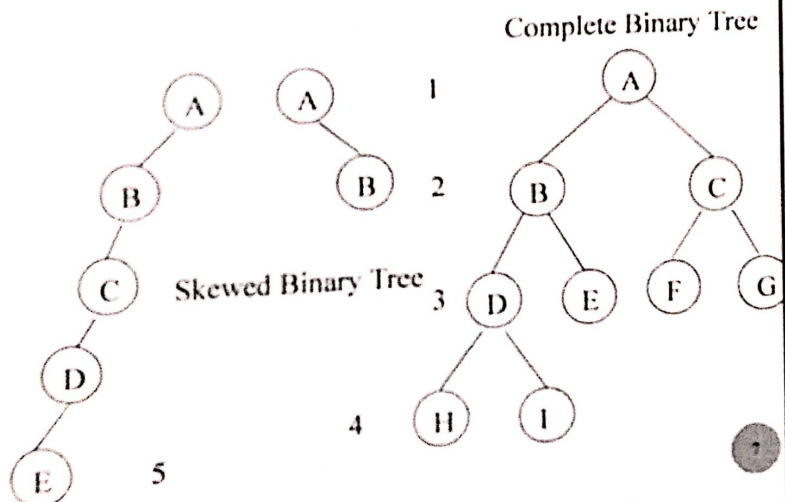
5

Binary Trees

- A binary tree is a finite set of nodes that is either empty or consists of a root and two disjoint binary trees called *the left subtree* and *the right subtree*.

6

Samples of Trees



7

Maximum Number of Nodes in Binary Tree

- The maximum number of nodes on level i of a binary tree is 2^{i-1} , $i \geq 1$.
- The maximum number of nodes in a binary tree of depth k is $2^k - 1$, $k \geq 1$.

$$\sum_{i=1}^k 2^{i-1} = 2^k - 1$$

8

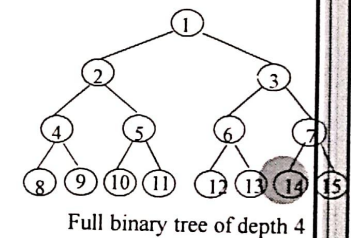
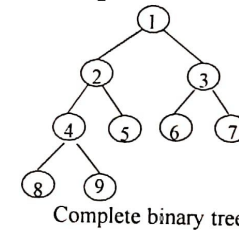
Relations between Number of Leaf Nodes and Nodes of Degree 2

For any nonempty binary tree, T , if n_0 is the number of leaf nodes and n_2 the number of nodes of degree 2, then $n_0 = n_2 + 1$

9

Full Binary Tree VS Complete Binary Tree

- A full binary tree of depth k is a binary tree of depth k having $2^{k+1} - 1$ nodes, $k \geq 1$.
- A binary tree with n nodes and depth k is complete iff its nodes correspond to the nodes numbered/labelled from 1 to n in the full binary tree of depth k .



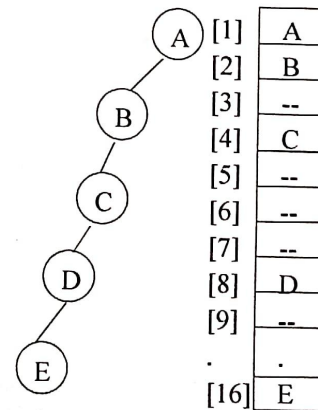
Binary Tree Representations

If a complete binary tree with n nodes is represented sequentially, then for any node with index i , $1 \leq i \leq n$, we have:

- $parent(i)$ is at $\text{floor}(i/2)$ if $i \neq 1$. If $i = 1$, i is at the root and has no parent.
- $left_child(i)$ is at $2i$ if $2i \leq n$. If $2i > n$, then i has no left child.
- $right_child(i)$ is at $2i+1$ if $2i+1 \leq n$. If $2i+1 > n$, then i has no right child.

11

Sequential Representation



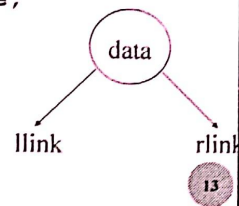
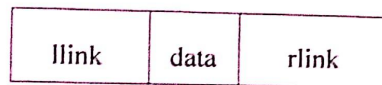
* waste space

* insertion/deletion problem

12

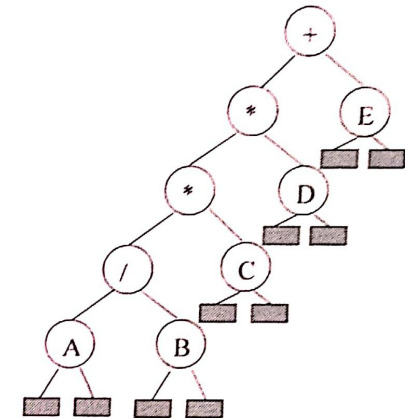
Linked Representation

```
struct treenode {
    struct treenode *llink;
    char data;
    struct treenode *rlink;
};
typedef struct treenode node;
```



13

Arithmetic Expression Using Binary Tree



inorder traversal
A / B * C * D + E
infix expression
preorder traversal
+ * * / A B C D E
prefix expression
postorder traversal
A B / C * D * E +
postfix expression

14

NODE STRUCTURE

```
struct treenode {
    struct treenode *llink;
    char data;
    struct treenode *rlink;
};
typedef struct treenode node;
```

15

Inorder Traversal A / B * C * D + E (recursive version)

```
void rinorder(node *t)
/* inorder tree traversal */
{
    if (t!=NULL) {
        rinorder(t->llink);
        printf("%c", t->data);
        rinorder(t->rlink);
    }
}
```

16

Preorder Traversal + ** / A B C D E (recursive version)

```
void rpreorder(node *t)
/* preorder tree traversal */
{
    if (t!=NULL) {
        printf("%c", t->data);
        rpreorder(t->llink);
        rpreorder(t->rlink);
    }
}
```

17

Postorder Traversal A B / C * D * E + (recursive version)

```
void rpostorder(node *t)
/* postorder tree traversal */
{
    if (t!=NULL) {
        rpostorder(t->llink);
        rpostorder(t->rlink);
        printf("%c", t->data);
    }
}
```

18

```
int inorder(node *t)
{ int i=-1;
  node *stack[20];
  while(1) {
      while(t!=NULL){
          i++;
          if (i==20) {
              printf("Stack full");
              return 1;
          }
          stack[i]=t;
          t=t->llink;
      }
      if(i== -1) return 0;
      t=stack[i];
      i--;
      printf("%c", t->data);
      t=t->rlink;
  }
}
```

19

```
int preorder(node *t)
{ int i=-1;
  node *stack[20];
  while(1)
      { while(t!=NULL){
          printf("%c", t->data);
          i++;
          if (i==20){
              printf("Stack full");
              return 1;
          }
          stack[i]=t;
          t=t->llink;
      }
      if(i== -1) return 0;
      t=stack[i];
      i--;
      t=t->rlink;
  }
}
```

20

```

int postorder(node *t)
{ int i=-1;
  node *stack[20];
  static int flag[20];
  while(1) {
    while(t!=NULL) {
      i++;
      if (i==20) {
        printf("Stack full");
        return 1;
      }
      stack[i]=t; flag[i]=0;
      t=t->llink;
    }
    if(i==-1) return 0;
    if(flag[i]==1) {
      printf("%c", stack[i]->data);
      i--;
    }
    else
    { t=stack[i];
      t=t->rlink;
      flag[i]=1;
    }
  }
}

```

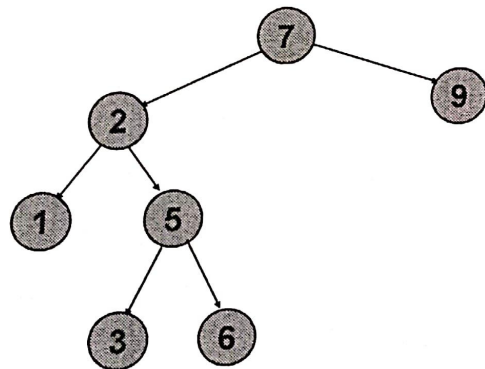
21

Binary search tree

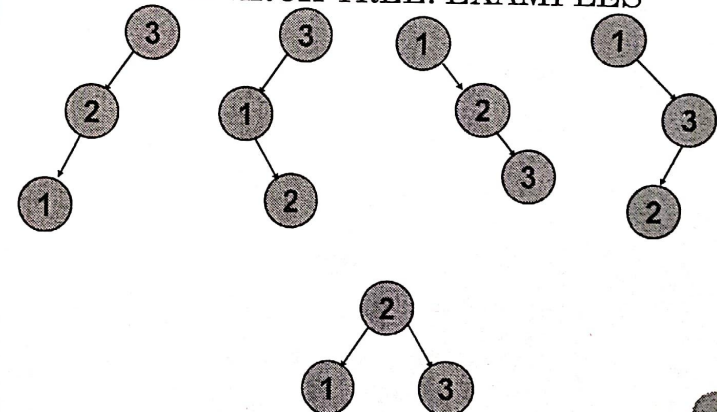
- Every element has a unique key.
- The keys in a nonempty left subtree (right subtree) are smaller (larger) than the key in the root.
- The left and right subtrees are also binary search trees.

22

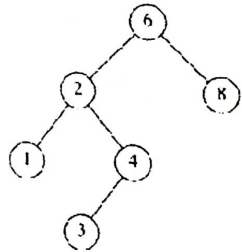
BINARY SEARCH TREE: EXAMPLES



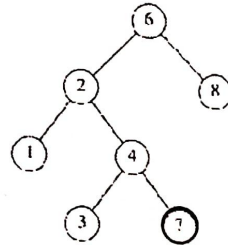
BINARY SEARCH TREE: EXAMPLES



BINARY SEARCH TREES



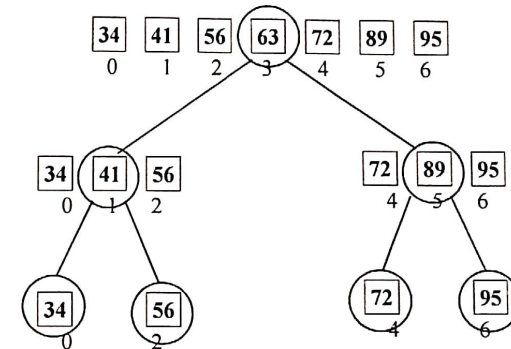
A binary search tree



Not a binary search tree

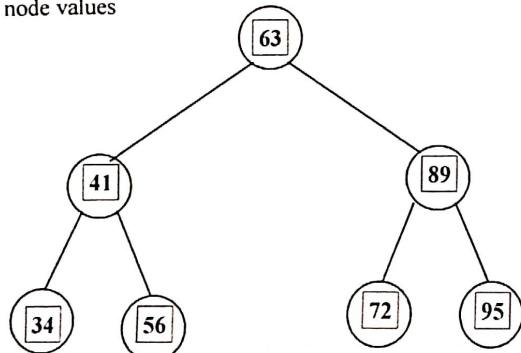
BINARY SEARCH ALGORITHM

Binary Search algorithm of an array of *sorted* items reduces the search space by one half after each comparison



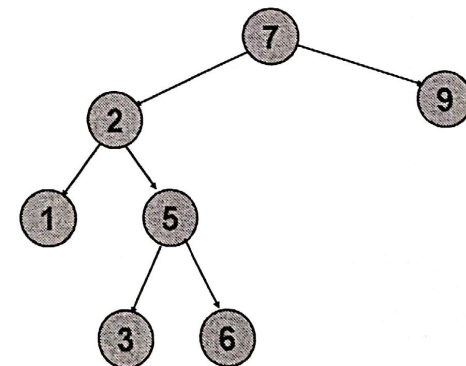
Organization rule for Binary Search Tree (BST)

- the values in all nodes in the left subtree of a node are less than the node value
- the values in all nodes in the right subtree of a node are greater than the node values



FIND

- You are given an element to find in a BST. If it exists, return the node. If not, return null.



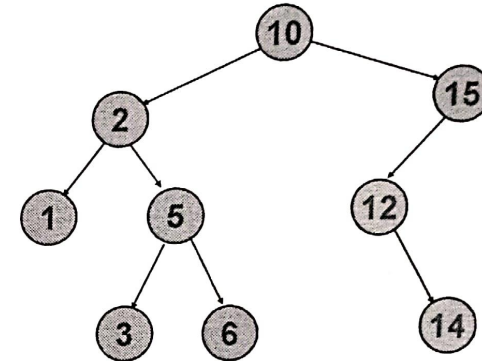
Search in a BST: C code

```
node *search(node *root, char key)
{
    /* return a pointer to the node that
       contains key. If there is no such
       node, return NULL */

    if (root==NULL) return NULL;
    if (key == root->data) return root;
    if (key < root->data)
        return search(root->llink, key);
    return search(root->rlink, key);
}
```

INSERTION: PRINCIPLE

When inserting a new element into a binary search tree, it will always become a leaf node.



```
node *insert(node *t, node *z)
{ // t contains the address of the root node if it exists
  // z contains the address of the node to be inserted
  node *y=NULL;
  node *x=t;
  while (x!=NULL){
      y=x;
      if (z->data < x->data)
          x=x->llink;
      else x=x->rlink;
  }

  if (y==NULL)
      t=z;
  else if (z->data < y->data)
      y->llink=z;
  else y->rlink=z;
  return t;
}
```

Removing An Element: Steps

- ❖ If the node is a leaf, simply delete it.
- ❖ If the node has one child, adjust parent's child reference to bypass the node.
- ❖ If the node has two children:

Replace the node with the node having the smallest element in the right subtree and remove that node from the right subtree

or

Replace the node with the node having the largest element in the left subtree and remove that node from the left subtree

```

typedef struct treeNode {
    int data;
    struct treeNode *left;
    struct treeNode *right;
} treeNode;

treeNode* FindMinPar(treeNode *node)
{
    treeNode *par=node;
    if(node->left==NULL) return NULL;
    else
    {
        while(node->left->left)
        {
            par=node->left;
            node=node->left;
        }

        return par;
    }
}

```

```

treeNode * Delete(treeNode *node, int data)
{
    treeNode *temp, *temp1, *right, *left;

    if(node==NULL)
    {
        printf("Element Not Found");
    }
    else if(data < node->data)
    {
        node->left = Delete(node->left, data);
    }
    else if(data > node->data)
    {
        node->right = Delete(node->right, data);
    }
}

```

```

else
{
    /* Now we can delete this node and replace with either minimum element
    in the right sub tree */
    if(node->right && node->left)
    {
        temp=node;
        /* Here we will replace with minimum element in the right sub tree */
        temp1 = FindMinPar(node->right);
        if(temp1==NULL)
        {
            left=node->left; node=node->right; node->left=left;
        }
        else
        {
            right=temp1->left->right; temp1->left->left=node->left;
            temp1->left->right=node->right; node=temp1->left;
            temp1->left=right;
        }

        free(temp);
    }
}

```

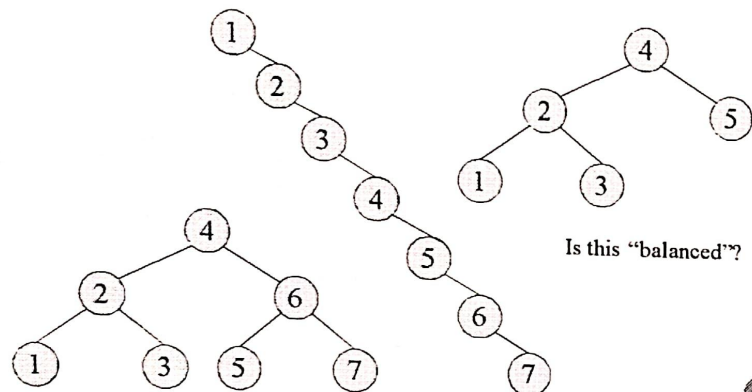
```

else
{
    /* If there is only one or zero children then we can directly
    remove it from the tree and connect its parent to its child */
    temp = node;
    if(node->left == NULL)
        node = node->right;
    else if(node->right == NULL)
        node = node->left;
    free(temp); /* temp is longer required */
}

return node;
}

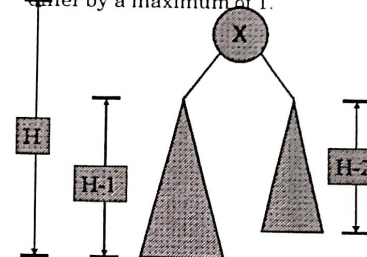
```

Balanced and unbalanced BST

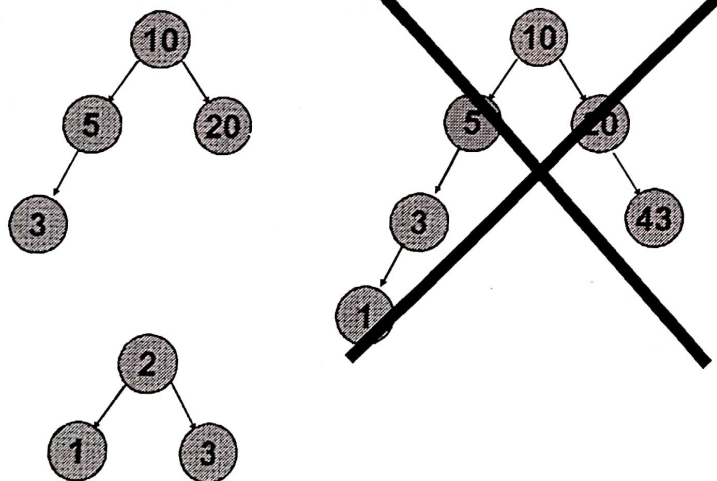


AVL Trees

- ❖ Unbalanced Binary Search Trees are bad. Worst case: operations take $O(n)$.
- ❖ The AVL tree is named after its two Soviet inventors, G. M. Adelson-Velskii and E. M. Landis, who published it in their 1962 paper "An algorithm for the organization of information".
- ❖ AVL (Adelson-Velskii & Landis) trees maintain height balance.
- ❖ For each node in tree, height of left subtree and height of right subtree differ by a maximum of 1.



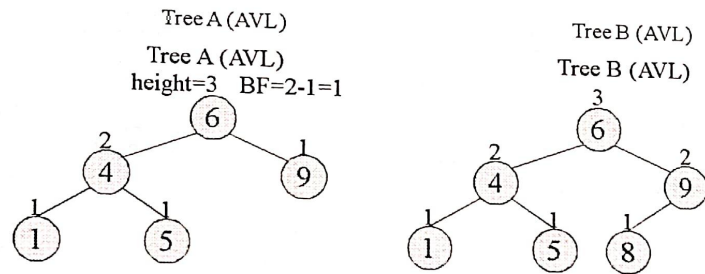
AVL Trees



AVL

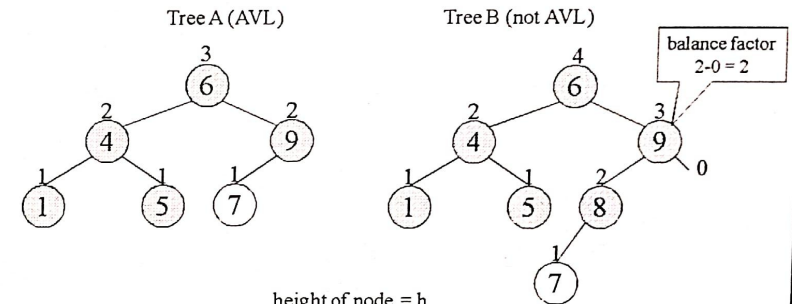
- ❖ AVL trees are height-balanced binary search trees
- ❖ Balance factor of a node
 $\text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$
- ❖ An AVL tree has balance factor calculated at every node
 For every node, heights of left and right subtree can differ by no more than 1

Node Heights



height of node = h
balance factor = $h_{\text{left}} - h_{\text{right}}$
empty height = 0

Node Heights after Insert 7



height of node = h
balance factor = $h_{\text{left}} - h_{\text{right}}$
empty height = 0

balance factor
 $2 - 0 = 2$

Insert and Rotation in AVL Trees

- ❖ Insert operation may cause balance factor to become 2 or -2 for some node
 - only nodes on the path from insertion point to root node have possibly changed in height
 - So after the Insert, go back up to the root node by node, updating heights
 - If a new balance factor (the difference $h_{\text{left}} - h_{\text{right}}$) is 2 or -2, adjust tree by *rotation* around the node

Insertions in AVL Trees

Let the node that needs rebalancing be α .

There are 4 cases:

Outside Cases (require single rotation) :

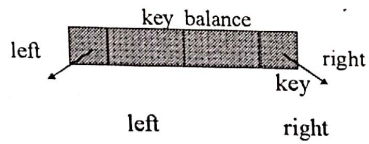
1. Insertion into left subtree of left child of α .
2. Insertion into right subtree of right child of α .

Inside Cases (require double rotation) :

3. Insertion into right subtree of left child of α .
4. Insertion into left subtree of right child of α .

The rebalancing is performed through four separate rotation algorithms.

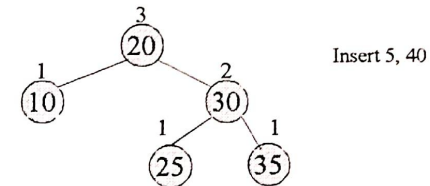
Implementation



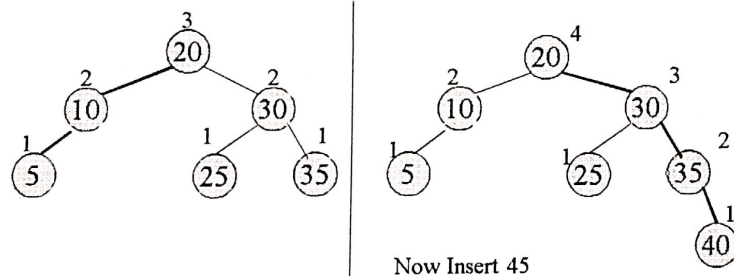
No need to keep the height; just the difference in height, i.e. the balance factor; this has to be modified on the path of insertion even if you don't perform rotations

Once you have performed a rotation (single or double) you won't need to go back up the tree

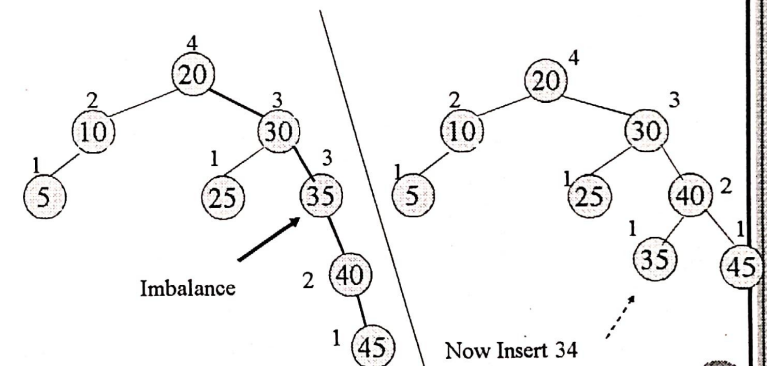
Example of Insertions in an AVL Tree



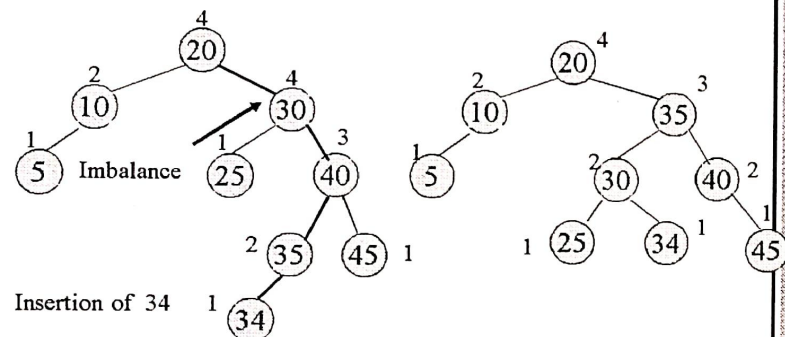
Example of Insertions in an AVL Tree



Single rotation (outside case)

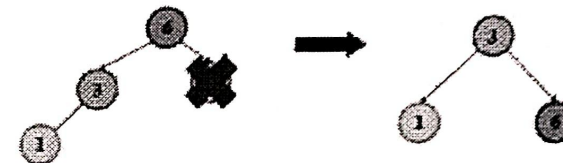


Double rotation (inside case)

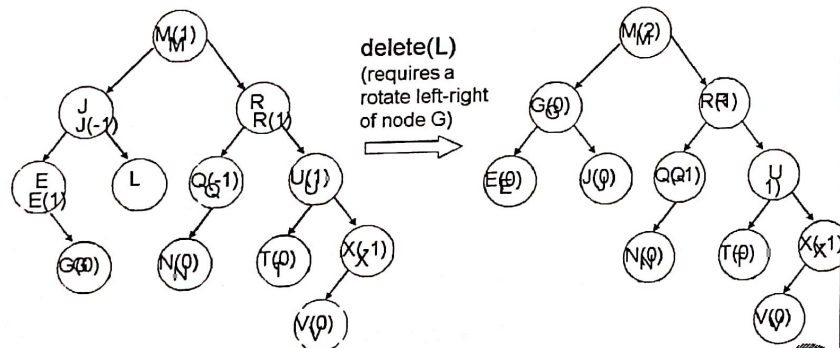


AVL Tree Deletion

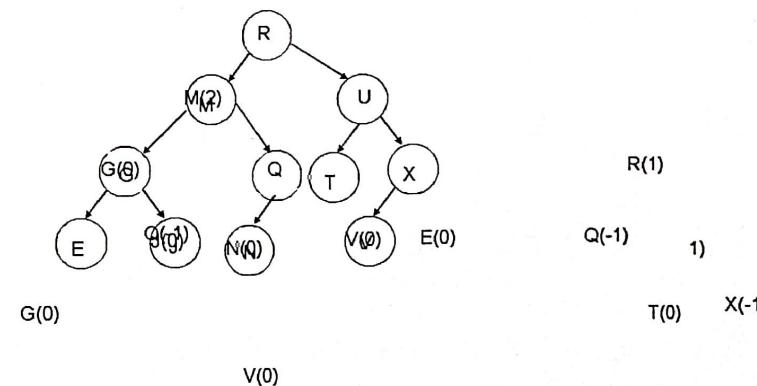
- Similar to insertion: do the delete and then rebalance
 - Rotations and double rotations
 - Imbalance may propagate upward so rotations at multiple nodes along path to root may be needed (unlike with insert)
- Simple example: a deletion on the right causes the left-left grandchild to be too tall
 - Call this the *left-left case*, despite deletion on the right
 - insert(6) insert(3) insert(7) insert(1) delete(7)



AVL Tree Delete



Notice that even after fixing J, M is still out of balance



Pros and Cons of AVL Trees

Arguments for AVL trees:

1. Search is $O(\log N)$ since AVL trees are always balanced.
2. Insertion and deletions are also $O(\log n)$
3. The height balancing adds no more than a constant factor to the speed of insertion.

Arguments against using AVL trees:

1. Difficult to program & debug; more space for balance factor.
2. Asymptotically faster but rebalancing costs time.
3. Most large searches are done in database systems on disk and use other structures (e.g. B-trees).